Credit Card Approval Prediction

Machine Learning–Based Credit Card Approval System

Project Description

Banks and financial institutions receive thousands of credit card applications every day. A significant portion of these are rejected due to factors such as high existing loan balances, insufficient income levels, or excessive credit inquiries. Manually reviewing each application is both time-consuming and highly prone to human error, making it an inefficient process at scale.

This project automates the credit card approval decision using machine learning. A predictive model is trained on historical applicant data and evaluates financial and demographic inputs — such as income type, employment duration, monthly income, and credit score — to determine whether an applicant is likely to be approved or rejected, similar to how real banks assess applications.

The trained model is saved as credit_card_model.pkl and integrated into a Flask web application, giving users an intuitive interface to enter applicant details and instantly receive a prediction. The interface is built with HTML/CSS and served through Flask's routing and templating system.

Scenarios

- **Scenario 1 — Automated Credit Card Application Screening:** A bank credit analyst enters a new applicant's financial profile — including income type, employment duration, and credit history — into the web application. The model returns an instant approval or rejection prediction, enabling the analyst to prioritize applications that require further review.
- **Scenario 2 — High-Risk Applicant Identification and Compliance Review:** A financial compliance officer uses the platform to screen applicants with past-due loan records. Feature engineering converts raw applicant attributes into a format the model can classify, flagging high-risk applicants as ineligible for a new credit card.
- **Scenario 3 — Customer Self-Service Credit Card Eligibility Check:** A prospective customer uses the web application to enter personal and financial details — income level, employment status, and credit history — before submitting a formal credit card application. The system instantly predicts the likelihood of approval, helping the customer understand their eligibility and reducing unnecessary application rejections.

Technical Architecture

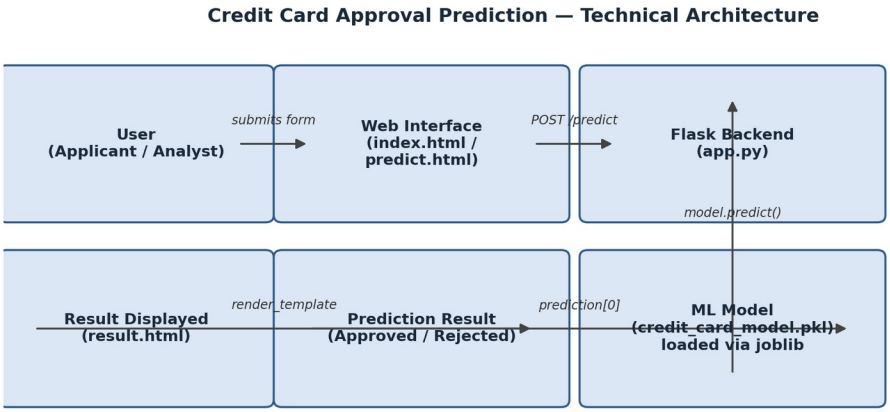


Figure 1: End-to-end flow from user input to prediction result

Description: The Credit Card Approval Prediction system follows a simple three-tier flow. The frontend (index.html, predict.html) collects applicant information through a web form. The Flask backend (app.py) receives the submitted values, converts them into a numeric feature array, and passes them to a pre-trained classification model loaded with joblib. The model's prediction is mapped to a human-readable outcome and rendered back to the user via result.html.

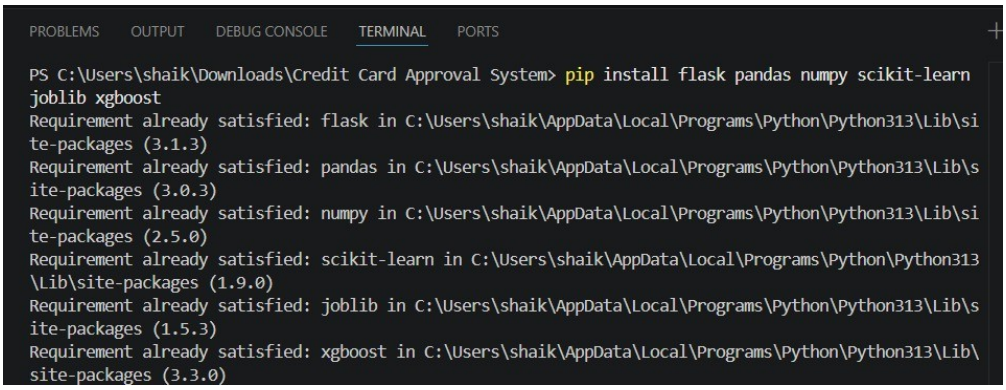
Pre-requisites

- Python Programming Proficiency: Python Documentation
- Flask Framework Knowledge: Flask Documentation
- Machine Learning Basics: scikit-learn Documentation
- Data Handling with pandas and numpy
- Model Persistence with joblib
- HTML, CSS Skills: for building predict.html, index.html, result.html and style.css
- Version Control with Git
- Development Environment: VS Code with an integrated terminal

Project Workflow

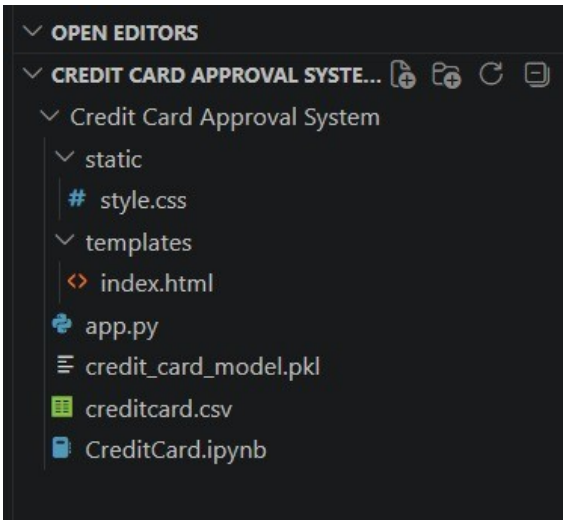
Milestone 1: Model Selection and Environment Setup

- **Activity 1.1:** Set up the development environment and install the required libraries — Flask, pandas, numpy, scikit-learn, joblib, and xgboost — using pip.



Installing project dependencies via pip

- **Activity 1.2:** Collect and organize the historical applicant dataset (creditcard.csv) containing financial and demographic fields used to train the model.
- **Activity 1.3:** Train and evaluate a classification model on the historical data inside a Jupyter notebook (CreditCard.ipynb), comparing candidate models on relevant metrics such as accuracy, precision, and recall.
- **Activity 1.4:** Save the best-performing model to disk using joblib as credit_card_model.pkl so it can be loaded directly by the Flask application without retraining.



Project folder structure: static/, templates/, app.py, credit_card_model.pkl, creditcard.csv, CreditCard.ipynb

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Define the feature set the model expects (e.g. income type, employment duration, monthly income, credit score) and establish how each will be captured from the web form.
- **Activity 2.2:** Implement feature preparation logic that converts submitted form values into a numeric array shaped to match the model's expected input using numpy.
- **Activity 2.3:** Load the trained model once at application startup with joblib.load(), avoiding the overhead of reloading the model on every request.

Milestone 3: app.py Development

Milestone 3 focuses on the core Flask application logic that ties the trained model to the web interface.

- **Activity 3.1:** Define the home route (/) which renders index.html, the landing page of the application.

- **Activity 3.2:** Define the /predict route to accept both GET and POST requests — GET renders the input form (predict.html), while POST reads the submitted form values, converts them to floats, reshapes them into a single-row feature array, and calls model.predict().
- **Activity 3.3:** Wrap the prediction logic in a try/except block so that any conversion or prediction errors are caught and surfaced back to the user through result.html rather than crashing the server.

```
from flask import Flask, render_template, request
import joblib
import numpy as np
app = Flask(__name__)

model = joblib.load("credit_card_model.pkl")

@app.route('/')
def home():
    return render_template("index.html")

@app.route('/predict', methods=['GET', 'POST'])
def predict():
    if request.method == 'POST':
        try:
            features = [float(x) for x in request.form.values()]
            final_features = np.array(features).reshape(1, -1)
            prediction = model.predict(final_features)

            if prediction[0] == 1:
                result = "Fraud Transaction"
            else:
                result = "Non-Fraud Transaction"

            return render_template("result.html", prediction=result)

        except Exception as e:
            return render_template("result.html", prediction=str(e))

    return render_template("predict.html")

if __name__ == "__main__":
    app.run(debug=True)
```

CreditCard.ipynb app.py

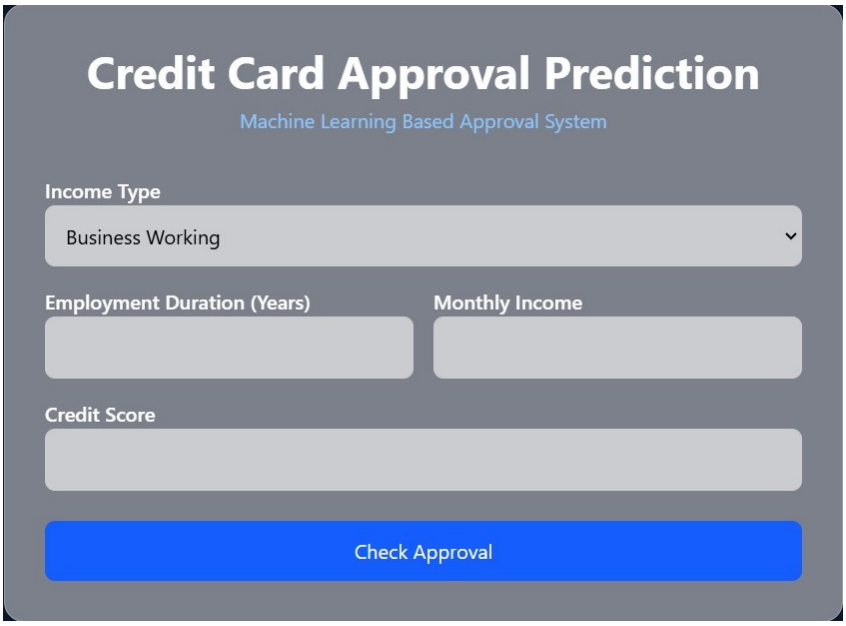
Credit Card Approval System > app.py > ...

```python
from flask import Flask, render_template, request
import joblib
import numpy as np
app = Flask(__name__)

model = joblib.load("credit_card_model.pkl")
@app.route('/')
def home():
    return render_template("index.html")

@app.route('/predict', methods=['GET', 'POST'])
def predict():

    if request.method == 'POST':

        try:

            features = [float(x) for x in request.form.values()]
            final_features = np.array(features).reshape(1, -1)
            prediction = model.predict(final_features)

            if prediction[0] == 1:
                result = "Fraud Transaction"
            else:
                result = "Non-Fraud Transaction"

            return render_template("result.html", prediction=result)

        except Exception as e:
            return render_template("result.html", prediction=str(e))

    return render_template("predict.html")


if __name__ == "__main__":
    app.run(debug=True)
```

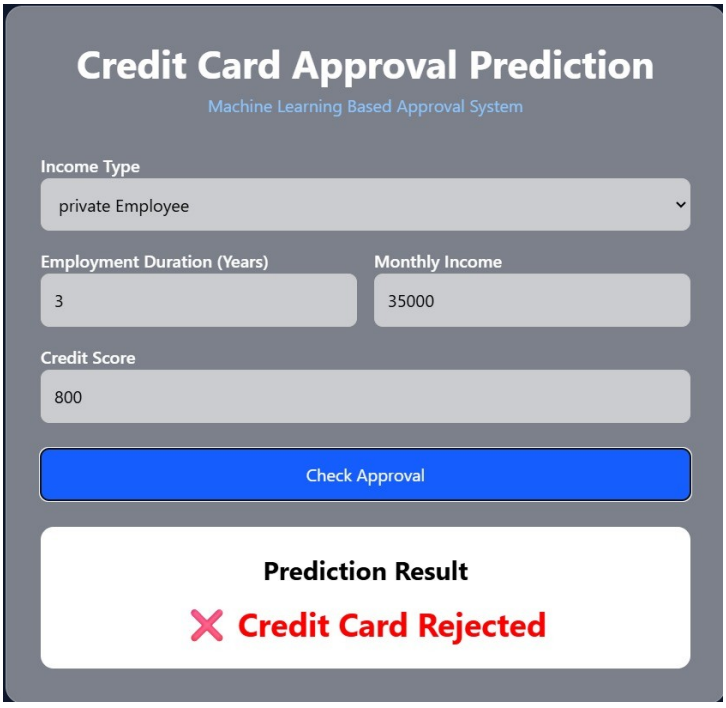
app.py — Flask routes, model loading, and prediction logic

Milestone 4: Frontend Development

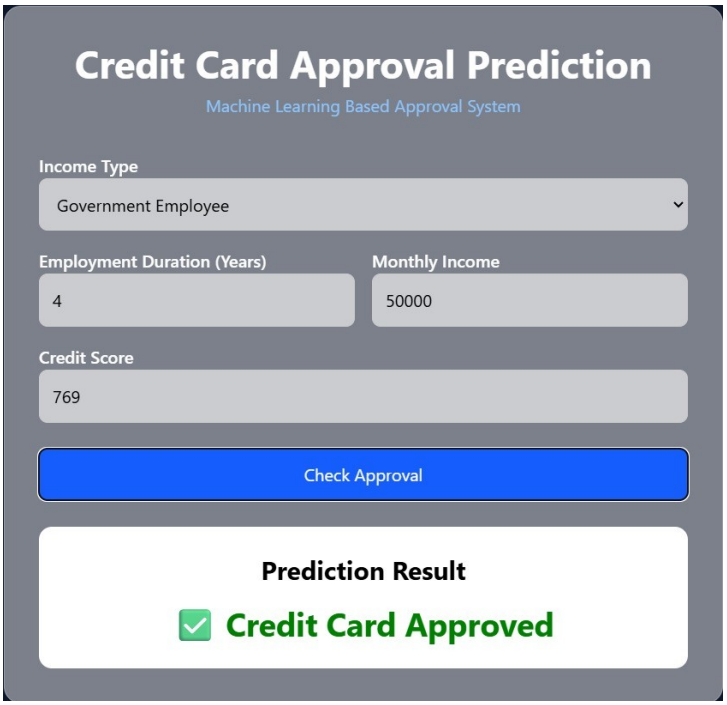
- **Activity 4.1:** Design index.html and predict.html with a clean, card-style layout capturing Income Type (dropdown), Employment Duration, Monthly Income, and Credit Score.
- **Activity 4.2:** Style the interface with static/style.css, using a dark-on-grey card design with a prominent "Check Approval" call-to-action button.
- **Activity 4.3:** Build result.html to clearly display the prediction outcome (Approved / Rejected) with a supporting icon and color cue (e.g. a red cross for a rejection).



Input form — Credit Card Approval Prediction page



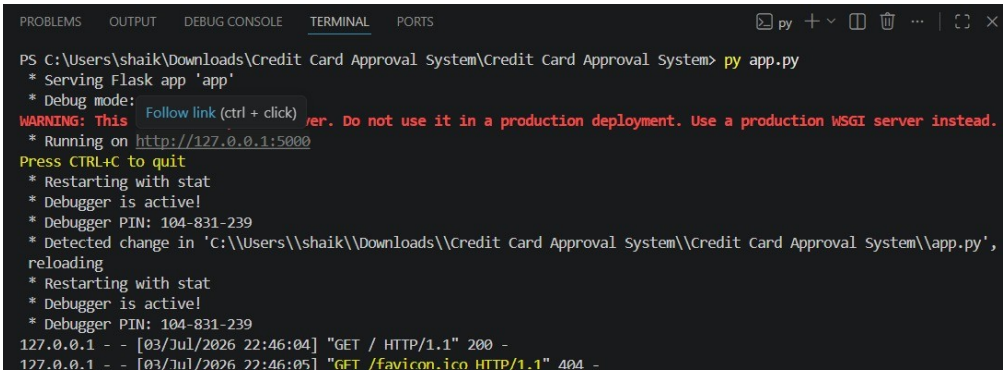
Filled form and prediction result — Credit Card Rejected



Filled form and prediction result — Credit Card Approved

Milestone 5: Local Testing and Deployment

- **Activity 5.1:** Install all dependencies from the project (Flask, pandas, numpy, scikit-learn, joblib, xgboost) and confirm they resolve without conflicts.
- **Activity 5.2:** Run the Flask development server locally with `py app.py` and confirm the app is served at http://127.0.0.1:5000, with debug mode and the auto-reloader active for iterative development.



Flask development server running locally on port 5000

- **Activity 5.3:** Test the application end-to-end in the browser: submit sample applicant profiles with varying income, employment duration, and credit score values, and confirm the model returns the expected approval or rejection outcome.

Note: The Flask development server (app.run(debug=True)) is intended for local testing only. For a production deployment, the application should be served through a production-grade WSGI server (e.g. Gunicorn or Waitress) behind a reverse proxy, or containerized and hosted on a cloud platform.

Conclusion

The Credit Card Approval Prediction system is a machine learning–powered Flask application that automates the initial screening of credit card applications. By training a classification model on historical applicant data and exposing it through a simple, intuitive web interface, the project reduces the manual effort and inconsistency involved in reviewing applications, while giving analysts, compliance officers, and prospective customers an instant, data-driven eligibility signal. Looking ahead, the project has room to grow — validating and expanding the model with more diverse applicant data, adding explainability (e.g. surfacing the key factors behind a decision), introducing user authentication and application history tracking, and moving from the Flask development server to a production-grade deployment (such as Gunicorn/Waitress behind a reverse proxy, Docker, or a managed ML hosting service like IBM Watson Machine Learning) so the system can reliably serve real-time predictions at scale.